BIG DATA ANALYTICS · PROJECT REPORT

Flight Delay Prediction and Analytics on 2015 US Domestic Flights Using MongoDB

MongoDB CRUD operations, partitioning and bucketing on 5,819,079 real flight records, with a step-by-step Docker and mongosh guide

Course	[Course name and code]
Team members	[Name, roll no.] [Name, roll no.] [Name, roll no.] [Name, roll no.]
Supervisor	[Supervisor name]
Institution	[Institution name]
Date	27 October 2026

All query outputs in this report were captured from the project's MongoDB 7 database (Docker container `2015flightdelaysandcancellations-mongo-1`) on 27 October 2026.

Contents

Abstract

How to read the command blocks in this report

1. Introduction and problem statement

2. Objectives

3. Dataset

4. System architecture

5. Database design

6. MongoDB CRUD operations

7. BDA application 1: Partitioning

8. BDA application 2: Bucketing

9. Machine learning model

10. Streaming layer (simulated velocity)

11. Results and discussion

12. Limitations

13. Conclusion and future scope

References

Appendix A: Viva quick sheet

Appendix B: Running everything as one script

Appendix C: Troubleshooting during the viva

Abstract

This project builds a complete Big Data pipeline for the Kaggle "2015 Flight Delays and Cancellations" dataset, which holds 5,819,079 US domestic flights. MongoDB 7 is the system of record. A streaming ingest job loads the CSV in batches of 10,000 documents without holding the dataset in memory. The schema denormalises reference data into each flight, pre-aggregates dashboard statistics with `$group` and `$merge` (the computed pattern), and uses compound and partial indexes. The report demonstrates the four MongoDB CRUD operations on real documents and two Big Data Analytics applications. The first is partitioning: a time-based split that doubles as the leakage-safe machine-learning split, partition pruning through an index, and an analysis of a designed (not deployed) shard key. The second is bucketing with `$bucket` and `$bucketAuto`. A LightGBM classifier trained on January to September and tested on October to December predicts whether a flight will arrive 15 or more minutes late, using pre-departure information only. It reaches a test ROC-AUC of 0.6276, a modest figure that reflects the absence of weather and aircraft-rotation data. A Kafka replay of December demonstrates simulated velocity. Every figure cites its measured source.

How to read the command blocks in this report

Every runnable block in this report carries a label that says **where to type it**. There are only two places:

Label on the block	Where you type it	What the prompt looks like	What goes there
POWERSHELL	A Windows PowerShell terminal (or the VS Code terminal), opened in the project folder	`PS D:\Projects\2015 Flight Delays and Cancellations>`	`docker ...` commands only: start the container, open mongosh, copy files
MONGOSH	The MongoDB shell, *inside* the container, which you open with a Docker command	`flight_delay>`	MongoDB queries only: `db.flights.find(...)`, `show collections`, `var demo = ...`
OUTPUT	Nothing to type		What the screen should show. It was captured from the real database on 2026-10-27

The rule of thumb. Look at the prompt before you paste.

- If it starts with `PS`, you are in PowerShell. Only `docker` commands work there.
- If it ends with `flight_delay>`, you are in mongosh. Only MongoDB queries work there.
- To move from PowerShell into mongosh, run `docker compose exec mongo mongosh flight_delay` (Section 6.0). To go back, type `exit`.

What a mistake looks like (both errors were reproduced on this machine, so you will recognise them):

You typed	In	You get	Fix
`db.flights.countDocuments()`	PowerShell	`An expression was expected after '('.` (a PowerShell parser error)	Open mongosh first (Section 6.0, step 3)
`docker compose up -d mongo`	mongosh	`Uncaught: SyntaxError: Missing semicolon. (1:6)`	Type `exit`, then run the command in PowerShell

Paste one block at a time and wait for the prompt to come back before pasting the next. Multi-line blocks can be pasted whole: mongosh waits until every bracket is closed before it runs the statement, and may show a continuation marker at the start of each extra line, which is normal. Lines that start with `var` print nothing; that is also normal. If copying from the PDF ever breaks a line, every query is also in the plain-text file `docs/report/viva-runbook.mongosh.js`, in the same order, ready to copy from an editor.

1. Introduction and problem statement

Flight delays cost airlines and passengers time and money. The US Department of Transportation counts a flight as delayed when it arrives 15 or more minutes after its scheduled arrival time (PROJECT_OVERVIEW.md §1). This project addresses three questions:

1. **Prediction.** Given only what is known before departure (date, airline, airports, scheduled hour, distance), how likely is a flight to arrive late?
2. **Explanation.** Where and when do delays concentrate (airline, airport, route, month, hour), and what causes them?
3. **Delivery.** How can both be served at interactive speed from millions of records?

The dataset is too large to load into a browser or a naive in-memory program, and a full scan costs 4 to 7 s per query (source: `docs/EXPLAIN_RESULTS.md`, Q1-Q7 "Before"). The problem is therefore also an engineering one: how to store, index, pre-aggregate and partition the data so that analytics and predictions are fast, reproducible and free of target leakage.

2. Objectives

1. Ingest all 5,819,079 flights into MongoDB by streaming the CSV in fixed-size batches, with bounded memory and an audit record for each run.
2. Design a document schema and an index set that answer the dashboard's queries in under one second.
3. Demonstrate MongoDB CRUD operations on real data, mirroring the operations the application itself performs.
4. Demonstrate two Big Data Analytics applications: **partitioning** (time-based partitions, partition pruning, a horizontal-partitioning design) and **bucketing** (fixed-boundary and equal-frequency histograms).
5. Train and serve a leakage-free delay classifier evaluated on a time-based hold-out.
6. Demonstrate a speed layer (simulated velocity) that scores replayed flights with the same predictor as the API.

3. Dataset

Source. Kaggle, "2015 Flight Delays and Cancellations", derived from the US Bureau of Transportation Statistics On-Time Performance data (PROJECT_OVERVIEW.md §3). It consists of three files: `flights.csv`, `airlines.csv` and `airports.csv`.

Property	Value	Source
Raw `flights.csv` size	592.4 MB	`docs/BIGDATA_JUSTIFICATION.md` (file size, measured 2026-10-16)

Property	Value	Source
Flight rows read and inserted	5,819,079 (0 rejected)	`ingest_runs`, via `docs/BIGDATA_JUSTIFICATION.md`
Airlines	14	collection count at run time (2026-10-27)
Airports	322	collection count at run time (2026-10-27)
`flights` as uncompressed BSON	3,924.4 MB, average document 674 bytes	`$collStats`, `docs/SCHEMA.md`
`flights` on disk (WiredTiger, compressed)	903.3 MB	`docs/SCHEMA.md`
Time span	1 January to 31 December 2015	`ml/artifacts/metrics.json` (`trainRange`, `testRange`)
Flights with a delay-cause breakdown	1,063,439	`docs/SCHEMA.md`
Cancelled flights	89,884	`docs/PROJECT_WALKTHROUGH.md` §3.5

Fields used (source column groups from PROJECT_OVERVIEW.md §3):

Group	Source columns
Calendar	`YEAR, MONTH, DAY, DAY_OF_WEEK`
Carrier and route	`AIRLINE, FLIGHT_NUMBER, TAIL_NUMBER, ORIGIN_AIRPORT, DESTINATION_AIRPORT`
Schedule and actual times	`SCHEDULED_DEPARTURE, DEPARTURE_TIME, DEPARTURE_DELAY, SCHEDULED_ARRIVAL, ARRIVAL_TIME, ARRIVAL_DELAY, SCHEDULED_TIME, ELAPSED_TIME, AIR_TIME, DISTANCE, TAXI_OUT, TAXI_IN`
Status	`CANCELLED, CANCELLATION_REASON, DIVERTED`
Delay causes (minutes; filled only for flights 15+ minutes late)	`AIR_SYSTEM_DELAY, SECURITY_DELAY, AIRLINE_DELAY, LATE_AIRCRAFT_DELAY, WEATHER_DELAY`

Data-quality issues handled. Times are `HHMM` integers, and `2400` occurs; they are converted to minutes from midnight and an hour of day. Blank cause columns are stored as `null`, never as 0. In October, 486,165 rows use 5-digit BTS airport ids instead of IATA codes. These rows are kept and flagged, then resolved offline: 483,018 of them (99.35%) were fully resolved (source: `docs/BIGDATA_JUSTIFICATION.md`, "Variety").

4. System architecture

The system follows a batch-plus-speed-layer design. The batch layer ingests the CSV, builds pre-aggregated statistics and model features, and trains the model offline. The speed layer replays December through Kafka. An Express API serves both to a React dashboard. Figure 1 shows the components and the data flow between them.

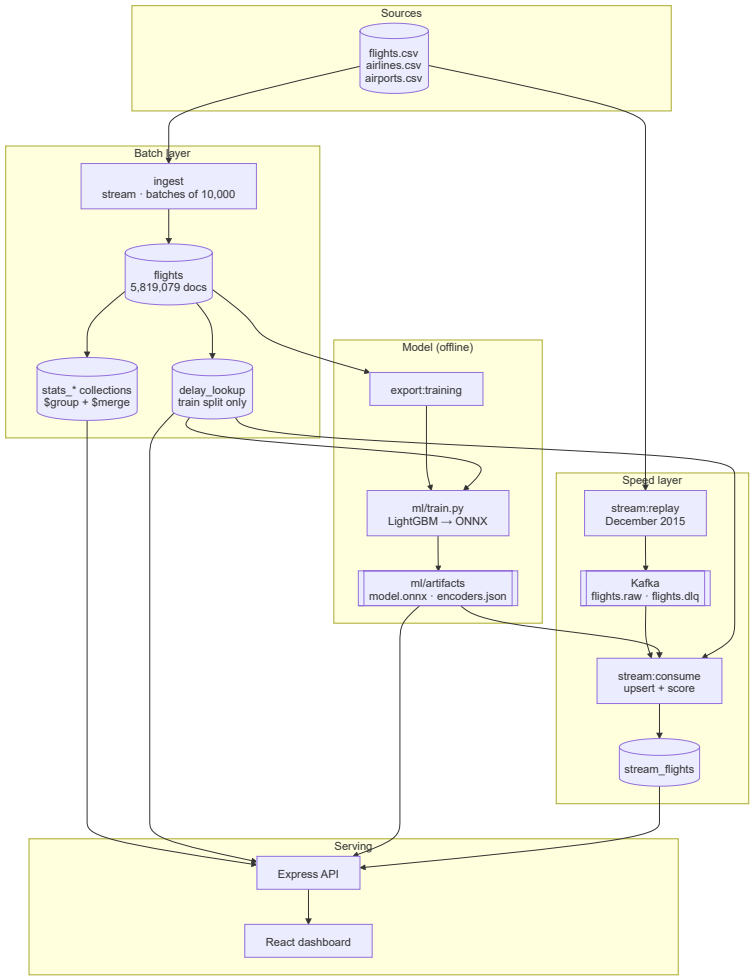


Figure 1. System architecture: batch layer, offline model, speed layer and serving layer (source diagram: `docs/diagrams/architecture-light.svg`, rendered from the Mermaid definition in `README.md`).

Component	Technology (source: `README.md`, "Tech stack")
Database	MongoDB 7, official `mongodb` Node.js driver (no ODM), in Docker
API	Node.js, Express 5, zod validation, TypeScript strict
Model training	Python, LightGBM, scikit-learn, exported to ONNX (`ml/train.py`)
Model serving	`onnxruntime-node` inside the API
Streaming	Apache Kafka (single KRaft node)
Dashboard	React, Vite, TypeScript

5. Database design

5.1 Collections

Document counts below were read from the live database on 2026-10-27, when the report queries were run.

Collection	Role	Documents
`flights`	System of record, one document per flight	5,819,079
`airlines`, `airports`	Reference data (`_id` = IATA code)	14, 322
`stats_daily`	Computed pattern: per date and airline	4,926
`stats_daily_origin`	Computed pattern: per date, origin and airline	386,704
`stats_route`	Computed pattern: per route	4,695
`stats_airport_hour`	Computed pattern: per airport and hour	4,259
`stats_cause_monthly`	Computed pattern: cause minutes per month and airline	162
`stats_cancellation`	Computed pattern: cancellations per month, airline and reason	440
`stats_delay_buckets`	Computed pattern: arrival-delay histogram (`$bucket`)	8
`stats_dow_hour`	Computed pattern: day-of-week by hour heatmap	168
`delay_lookup_cells`	Intermediate cells for model features (train split)	249,130
`delay_lookup`	Smoothed historical delay rates (train split)	36,420
`model_runs`	Model registry	2
`ingest_runs`	Ingest audit records	17
`stream_flights`	Speed layer: replayed December flights with live scores	479,230

5.2 The `flights` document

Each flight document groups its fields as follows (source: `docs/SCHEMA.md`, interface `FlightDoc` in `server/src/db/collections.ts`):

Group	Fields
Calendar	`date`, `year`, `month`, `day`, `dayOfWeek`
Carrier	`airline`, **`airlineName`**, `flightNumber`, `tailNumber`
Origin and destination	`origin`, `dest`, **`originCity`**, **`originState`**, **`destCity`**, **`destState`**, plus flags for derived airport codes
Route	**`route`** (`"ORIGIN-DEST"`)
Schedule	`schedDepMin`, `schedDepHour`, `schedArrMin`, `distance`, `scheduledTime`
Outcome (post-departure)	`depDelay`, `arrDelay`, `taxiOut`, `taxiIn`, `airTime`, `elapsedTime`, `cancelled`, `cancellationReason`, `diverted`
Label	`isDelayed` = `arrDelay >= 15`; `null` when cancelled, diverted or unknown
Causes	`causes: { carrier, weather, nas, security, lateAircraft }` or `null`
Split	`split`: `"train"` (months 1-9) or `"test"` (10-12), set at ingest

Fields in bold are denormalised or derived at ingest.

5.3 Design choices and their reasons

Choice	Alternative	Reason	Cost
Denormalise airline name, origin and destination city and state, and `route` into every flight	Normalised flights with `$lookup` at read time	A `$lookup` runs once per input document in a 5.8-million-row aggregation, and it adds a cross-collection step on a sharded cluster. The reference data is tiny (14 + 322 documents) and fixed for 2015, so update anomalies are not a practical risk. There is no `$lookup` anywhere in `server/src`.	About 103 of the 674 average bytes per document, roughly 15% (`docs/SCHEMA.md`)
Computed pattern: `stats_*` collections built with `$group` + `$merge`	Aggregate `flights` on every request	A whole-collection route ranking takes 13,499 ms (Q8). The same ranking read from `stats_route` takes 38 ms median over HTTP (`docs/EXPLAIN_RESULTS.md`).	The data is only as fresh as the last rebuild, which took 110.2-370.8 s over seven runs (`docs/SCHEMA.md`)
`$merge` with a `buildId` sweep instead of `$out`	`$out` replaces the whole collection	`$merge` upserts by `_id`, so a collection is never empty during a build. The sweep then deletes keys that no longer exist (see D2, Section 6.4).	One extra `deleteMany` per collection
Nullable `causes` sub-document	Five nullable columns or a child table	The absence of causes is a single `null`, which enables a small partial index	Queries must handle `null` explicitly
Time-based `split` stored on each flight	Random split	A random split leaks seasonality into training and overstates AUC (PROJECT_OVERVIEW.md §7)	The test period differs from training (Section 9)

5.4 Indexes on `flights`

Index sizes and timings are from `docs/SCHEMA.md` and `docs/EXPLAIN_RESULTS.md`. Timings are server-side `executionTimeMillis`, collection scan → indexed plan.

Index	Key	Size	Serves	Evidence
`_id_`	`{ _id: 1 }`	64.7 MB	Keyset pagination in `/api/flights`	December window: 8.8 s → 21 ms with `_id` bounds
`date_1`	`{ date: 1 }`	29.0 MB	Date-range reads	Q7: 4,392 → 66 ms
`airline_1_date_1`	`{ airline: 1, date: 1 }`	50.8 MB	One airline over a date range (equality, then range)	Q1: 7,037 → 107 ms
`origin_1_schedDepHour_1`	`{ origin: 1, schedDepHour: 1 }`	39.7 MB	One airport, optionally by hour	Q2: 4,614 → 39 ms

Index	Key	Size	Serves	Evidence
`route_1`	`{ route: 1 }`	48.0 MB	One route	Q4: 4,953 → 178 ms
`isDelayed_1_month_1`	`{ isDelayed: 1, month: 1 }`	28.1 MB	Delayed counts per month, answered from the index alone	Q6: 6,280 → 33 ms, 0 documents examined
`month_1_airline_1_with_causes`	`{ month: 1, airline: 1 }`, **partial** on `{ causes: { $type: "object" } }`	6.0 MB	Cause analysis	Q5: 4,653 → 403 ms
`airline_1_origin_1__id_1`	`{ airline: 1, origin: 1, _id: 1 }`	100.9 MB	`/api/flights?airline=&origin=` in page order	HA + ATL: 7.5 s → 5 ms

Why a partial index. Only 1,063,439 of 5,819,079 flights (18.3%) carry a `causes` object. Indexing only those flights keeps the index at 6.0 MB, against 28.1-50.8 MB for the full-collection indexes. Queries must repeat `causes: { $type: "object" }`, or the planner cannot use the index (`docs/SCHEMA.md`).

Indexes deliberately not added. These are `dest`, `split` (low selectivity) and `originIsIata`. Every index costs memory and slows every insert (`docs/SCHEMA.md`, "Considered and not added").

6. MongoDB CRUD operations

6.0 Getting into mongosh (Docker)

MongoDB runs in a Docker container defined in `docker-compose.yml` (image `mongo:7`, service `mongo`, container `2015flightdelaysandcancellations-mongo-1`, port 27017 bound to localhost). Docker Desktop must be running.

Step 1. Open PowerShell in the project folder.

▶ POWERSHELL · type at the prompt PS D:\Projects\2015 Flight Delays and Cancellations>

```
cd "D:\Projects\2015 Flight Delays and Cancellations"
```

Step 2. Start the container and check that it is healthy.

▶ POWERSHELL · type at the prompt PS D:\Projects\2015 Flight Delays and Cancellations>

```
docker compose up -d mongo
docker compose ps
```

OUTPUT · what you should see (nothing to type)

```
NAME                                        IMAGE     COMMAND                  SERVICE   CREATED
STATUS                    PORTS
2015flightdelaysandcancellations-mongo-1   mongo:7   "docker-entrypoint.s…"   mongo     24 hours
ago   Up 20 minutes (healthy)   127.0.0.1:27017->27017/tcp
```

The compose file has a health check that pings MongoDB every 10 s, so wait for `(healthy)` in the STATUS column. CREATED and the uptime will differ on your machine. The two `docker compose` commands only work from the project folder (Step 1), where `docker-compose.yml` is.

Step 3. Open mongosh inside the container, connected to the project database.

▶ POWERSHELL · type at the prompt PS D:\Projects\2015 Flight Delays and Cancellations>

```
docker compose exec mongo mongosh flight_delay
```

`docker compose exec mongo` runs a program inside the running `mongo` service; `mongosh flight_delay` is that program, the MongoDB shell, opened on the `flight_delay` database. The prompt changes from `PS ...>` to `flight_delay>`. **From here until you type `exit`, you are in mongosh.** (An equivalent that works from any folder: `docker exec -it 2015flightdelaysandcancellations-mongo-1 mongosh flight_delay`.)

Step 4. First check: the right database, fully loaded.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.getName()
db.flights.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
flight_delay
5819079
```

Step 5. Prepare the throwaway database for the write examples.

The real database, `flight_delay`, is the system of record that the API and dashboard read, so the write examples never touch it. They write to a separate database, `flight_delay_demo`, filled with *copies of real documents*. `var demo` is a handle to it. `dropDatabase()` empties it first, so the whole sequence can be repeated from here at any time.

▶ MONGOSH · type at the prompt `flight_delay>` (inside the container)

```
var demo = db.getSiblingDB('flight_delay_demo')
demo.dropDatabase()
```

OUTPUT · what you should see (nothing to type)

```
{ ok: 1, dropped: 'flight_delay_demo' }
```

Operation	MongoDB methods shown	Runs against	The same operation in the project code
Create	`$out`, `insertOne`, `insertMany`	`flight_delay_demo`	`server/src/ingest/run.ts`: CSV stream → `insertMany` in batches of 10,000, `ordered: false`
Read	`find` + projection/sort/limit, `explain`, `countDocuments`, `aggregate`	`flight_delay` (read only)	`server/src/analytics/flights.ts` (`/api/flights`), `server/src/stats/builders.ts`
Update	`updateMany` (pipeline update), `updateOne` with `upsert`	`flight_delay_demo`	`server/src/stream/store.ts` (Kafka consumer upsert), `server/src/ingest/resolve.ts`
Delete	`deleteMany`	`flight_delay_demo`	`server/src/stats/build.ts`, `server/src/features/lookup.ts` (stale-build sweep)

6.1 Create

C1. Copy a real slice with `$out`. All ATL departures on 25 December 2015 are copied server-side into the demo database. No document passes through the shell.

▶ MONGOSH · type at the prompt `flight_delay>` (inside the container)

```
db.flights.aggregate([
  { $match: { date: ISODate('2015-12-25'), origin: 'ATL' } },
  { $out: { db: 'flight_delay_demo', coll: 'flights' } }
])
demo.flights.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
700
```

C2. `insertOne`. `findOne` reads a real flight (United 1665, ORD-LAX, 1 January 2015, one that reports delay causes); `insertOne` stores a copy under a readable `_id`. The second `findOne` verifies it, showing only a few fields (a *projection*).

▶ MONGOSH · type at the prompt `flight_delay>` (inside the container)

```
var src = db.flights.findOne({ route: 'ORD-LAX', causes: { $type: 'object' } }, { _id: 0 })
demo.flights.insertOne({ ...src, _id: 'ORD-LAX-UA1665-2015-01-01' })
demo.flights.findOne(
  { _id: 'ORD-LAX-UA1665-2015-01-01' },
  { airline: 1, flightNumber: 1, route: 1, date: 1, arrDelay: 1 }
)
```

OUTPUT · what you should see (nothing to type)

```
{ acknowledged: true, insertedId: 'ORD-LAX-UA1665-2015-01-01' }
{
  _id: 'ORD-LAX-UA1665-2015-01-01',
  date: ISODate('2015-01-01T00:00:00.000Z'),
  airline: 'UA',
  flightNumber: 1665,
  route: 'ORD-LAX',
  arrDelay: 16
}
```

C3. `insertMany`. Three real JFK-LAX flights of 24 December 2015 are inserted in one call. `ordered: false` is the option the ingest job uses: one bad document does not stop the rest of the batch.

▶ MONGOSH · type at the prompt `flight_delay>` (inside the container)

```
var three = db.flights.find(
  { route: 'JFK-LAX', date: ISODate('2015-12-24') }, { _id: 0 }
).sort({ schedDepMin: 1 }).limit(3).toArray()
demo.flights.insertMany(three, { ordered: false })
demo.flights.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae05078960efd89a0750416'),
    '1': ObjectId('6ae05078960efd89a0750417'),
    '2': ObjectId('6ae05078960efd89a0750418')
  }
}
704
```

The ObjectIds are generated at insert time, so yours will differ. The count is 700 (C1) + 1 (C2) + 3 (C3) = 704.

In the project. `npm run ingest` streams `flights.csv` (592.4 MB) with `csv-parse`, validates each row, and calls `insertMany(batch, { ordered: false })` for every 10,000 rows. The next batch is read only after the insert returns, so memory stays bounded: 5,819,079 rows were inserted with 0 rejected, at a peak of 362 MB of Node.js memory (`ingest_runs`, via `docs/BIGDATA_JUSTIFICATION.md`).

6.2 Read

All reads run on the real `flight_delay` database.

R1. `find` with a filter, projection, sort and limit. American Airlines (`AA`) flights in March 2015, first three by date.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.flights.find(
  { airline: 'AA', date: { $gte: ISODate('2015-03-01'), $lt: ISODate('2015-04-01') } },
  { _id: 0, date: 1, flightNumber: 1, route: 1, arrDelay: 1 }
).sort({ date: 1 }).limit(3)
```

OUTPUT · what you should see (nothing to type)

```
[
  { date: ISODate('2015-03-01T00:00:00.000Z'), flightNumber: 2400, route: 'LAX-DFW', arrDelay: null
},
  { date: ISODate('2015-03-01T00:00:00.000Z'), flightNumber: 258, route: 'LAX-MIA', arrDelay: -14
},
  { date: ISODate('2015-03-01T00:00:00.000Z'), flightNumber: 1234, route: 'LAS-ORD', arrDelay: 35 }
]
```

(mongosh may print each document over several lines; the values are the same.) The `null` arrival delay is a flight with no recorded outcome.

R1b. Prove that an index answers R1 (`explain`). `explain('executionStats')` returns the plan MongoDB chose and what it touched. The second line picks out the four fields that matter. It starts with `;` so that mongosh does not join it to the previous line.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
var e = db.flights.find(
  { airline: 'AA', date: { $gte: ISODate('2015-03-01'), $lt: ISODate('2015-04-01') } }
).explain('executionStats')
;({
  index: e.queryPlanner.winningPlan.inputStage.indexName,
  returned: e.executionStats.nReturned,
  keysExamined: e.executionStats.totalKeysExamined,
  docsExamined: e.executionStats.totalDocsExamined
})
```

OUTPUT · what you should see (nothing to type)

```
{
  index: 'airline_1_date_1',
  returned: 45966,
  keysExamined: 45966,
  docsExamined: 45966
}
```

Keys examined = documents examined = documents returned: MongoDB touched only the 45,966 matching flights out of 5,819,079 (0.79%). The compound index `{ airline: 1, date: 1 }` serves the equality on `airline` first and then the range on `date`. As a forced collection scan, the same query took 7,037 ms, against 107 ms with the index (`docs/EXPLAIN_RESULTS.md`, Q1).

R2. `countDocuments`. Delayed flights in December.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.flights.countDocuments({ month: 12, isDelayed: true })
```

OUTPUT · what you should see (nothing to type)

```
96772
```

R3. Aggregation: December delay rate by airline, top five. `$match` keeps completed December flights (`isDelayed` is `null` for cancelled or diverted ones), `$group` counts per airline, `$project` computes the rate, `$sort` and `$limit` rank.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.flights.aggregate([
  { $match: { month: 12, isDelayed: { $ne: null } } },
  { $group: { _id: '$airline', flights: { $sum: 1 },
              delayed: { $sum: { $cond: ['$isDelayed', 1, 0] } } } },
  { $project: { flights: 1,
                delayRate: { $round: [{ $divide: ['$delayed', '$flights'] }, 4] } } },
  { $sort: { delayRate: -1 } },
  { $limit: 5 }
])
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: 'NK', flights: 10473, delayRate: 0.2995 },
  { _id: 'B6', flights: 23197, delayRate: 0.2945 },
  { _id: 'VX', flights: 5470, delayRate: 0.2812 },
  { _id: 'OO', flights: 45984, delayRate: 0.2453 },
  { _id: 'F9', flights: 8018, delayRate: 0.2406 }
]
```

Spirit (NK) and JetBlue (B6) had nearly 30% of December flights arrive 15+ minutes late.

R4. Read the pre-aggregated `stats_route` collection. The three most delay-prone routes with at least 1,000 flights, read from 4,695 pre-computed route documents instead of 5.8 million flights.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.stats_route.find(
  { flights: { $gte: 1000 } }, { flights: 1, delayRate: 1 }
).sort({ delayRate: -1 }).limit(3)
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: 'DCA-JFK', flights: 1027, delayRate: 0.3515881708652793 },
  { _id: 'ORD-COS', flights: 1027, delayRate: 0.33867735470941884 },
  { _id: 'LGA-CMH', flights: 1556, delayRate: 0.3326345213137666 }
]
```

In the project. `/api/analytics/delay-rate?by=route` reads `stats_route` this way: 38 ms median over HTTP, against 13,499 ms for the same ranking computed over `flights` (`docs/EXPLAIN_RESULTS.md`, Q8). `/api/flights` uses R1-style filters with a projection and keyset pagination capped at 100 rows (`server/src/analytics/flights.ts`).

6.3 Update

U1. `updateMany` with a pipeline update. A new field `delayBand` is derived from each document's own `isDelayed` and `arrDelay`. Passing an array (`[ { $set: ... } ]`) instead of a plain object makes it a *pipeline update*, which can read other fields of the same document. The `aggregate` after it verifies the result.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
demo.flights.updateMany({}, [
  { $set: { delayBand: { $switch: {
    branches: [
      { case: { $eq: ['$isDelayed', null] }, then: 'no-outcome' },
      { case: { $lt: ['$arrDelay', 15] }, then: 'on-time' },
      { case: { $lt: ['$arrDelay', 60] }, then: '15-59 min late' }
    ],
    default: '60+ min late'
  } } } }
])
demo.flights.aggregate([
  { $group: { _id: '$delayBand', n: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

OUTPUT · what you should see (nothing to type)

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 704,
  modifiedCount: 704,
  upsertedCount: 0
}
[
  { _id: '15-59 min late', n: 100 },
  { _id: '60+ min late', n: 34 },
  { _id: 'no-outcome', n: 14 },
  { _id: 'on-time', n: 556 }
]
```

All 704 documents were updated; the bands add up to 704.

U2. The idempotent upsert of the Kafka consumer. Kafka delivers each event *at least once*, so the same flight can arrive twice. The consumer therefore writes with one `updateOne` that is safe to repeat:

- `$setOnInsert` writes the flight and its prediction only when the document is first created;
- `$set` records when it was last seen;
- `$inc` counts deliveries;
- `upsert: true` inserts the document if it does not exist yet.

A real `stream_flights` document is copied to the demo database, then one redelivery is simulated.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
var s = db.stream_flights.findOne({ _id: '2015-12-01|AA|1230|SEA|5' })
demo.stream_flights.insertOne(s)
demo.stream_flights.findOne({ _id: s._id }, { deliveries: 1, 'prediction.label': 1 })
```

OUTPUT · what you should see (nothing to type)

```
{ acknowledged: true, insertedId: '2015-12-01|AA|1230|SEA|5' }
{
  _id: '2015-12-01|AA|1230|SEA|5',
  deliveries: 4,
  prediction: { label: 'on-time' }
}
```

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
demo.stream_flights.updateOne(
  { _id: s._id },
  { $setOnInsert: { runId: s.runId, pre: s.pre, outcome: s.outcome,
                    prediction: s.prediction },
    $set: { lastWrittenAt: new Date() },
    $inc: { deliveries: 1 } },
  { upsert: true }
)
demo.stream_flights.findOne({ _id: s._id }, { deliveries: 1, 'prediction.label': 1 })
demo.stream_flights.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  _id: '2015-12-01|AA|1230|SEA|5',
  deliveries: 5,
  prediction: { label: 'on-time' }
}
1
```

The redelivery matched the existing document (`upsertedCount: 0`), raised `deliveries` from 4 to 5, left the prediction unchanged, and created no duplicate (still 1 document). The starting value 4 is how often this flight was delivered in the project's own replays; if the stream is replayed again, it will be higher, and the point is that it rises by exactly 1.

In the project. `upsertEvents` in `server/src/stream/store.ts` sends exactly this update shape as an ordered `bulkWrite` . Over all of December, 479,230 events produced 479,230 stored documents for 479,230 distinct keys, and re-sending 5,000 events left the count at 479,230 (`docs/STREAMING.md`).

6.4 Delete

D1. `deleteMany` **with a filter.** Cancelled flights are removed from the demo copy. Counting before and after verifies the delete.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
demo.flights.countDocuments({ cancelled: true })
demo.flights.deleteMany({ cancelled: true })
demo.flights.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
10
{ acknowledged: true, deletedCount: 10 }
694
```

704 − 10 = 694. (This is a demonstration: the application keeps cancelled flights, because they feed the cancellation analysis.)

D2. The stale-build sweep. Each pre-aggregated document carries the `buildId` of the build that wrote it. After a rebuild, any document with an older `buildId` is a key that no longer exists, and is deleted. Here the real `stats_delay_buckets` (8 documents) is copied, one stale document is added, and the sweep removes exactly that one.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.stats_delay_buckets.aggregate([
  { $out: { db: 'flight_delay_demo', coll: 'stats_delay_buckets' } }
])
var current = demo.stats_delay_buckets.findOne().buildId
demo.stats_delay_buckets.insertOne({ _id: 'stale-key', flights: 1, buildId: 'old-build' })
demo.stats_delay_buckets.deleteMany({ buildId: { $ne: current } })
demo.stats_delay_buckets.countDocuments()
```

OUTPUT · what you should see (nothing to type)

```
{ acknowledged: true, insertedId: 'stale-key' }
{ acknowledged: true, deletedCount: 1 }
8
```

In the project. `npm run build:stats` (`server/src/stats/build.ts`) runs `deleteMany({ buildId: { $ne: buildId } })` after each `$merge`, and `npm run build:lookup` does the same for `delay_lookup`. Together with `$merge`, this makes every rebuild idempotent and keeps each collection non-empty while it is rebuilt.

7. BDA application 1: Partitioning

Partitioning divides a large dataset so that a query, a job or a machine handles only the part it needs. This project applies it at three levels: a logical **time partition** stored on every document, **partition pruning** through an index, and a **horizontal partition (sharding)** design checked against the real data.

7.1 P1: Time-based partitions (the leakage-safe ML split)

At ingest, `server/src/ingest/transform.ts` stores `split: 'train'` on January-September flights and `split: 'test'` on October-December flights, next to `month`. Every later job (`build:lookup`, `export:training`) filters on this one stored definition, so no test-period information can reach the model's training features.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.flights.aggregate([
  { $group: { _id: { split: '$split', month: '$month' }, flights: { $sum: 1 } } },
  { $sort: { '_id.month': 1 } }
])
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: { split: 'train', month: 1 }, flights: 469968 },
  { _id: { split: 'train', month: 2 }, flights: 429191 },
  { _id: { split: 'train', month: 3 }, flights: 504312 },
  { _id: { split: 'train', month: 4 }, flights: 485151 },
  { _id: { split: 'train', month: 5 }, flights: 496993 },
  { _id: { split: 'train', month: 6 }, flights: 503897 },
  { _id: { split: 'train', month: 7 }, flights: 520718 },
  { _id: { split: 'train', month: 8 }, flights: 510536 },
  { _id: { split: 'train', month: 9 }, flights: 464946 },
  { _id: { split: 'test', month: 10 }, flights: 486165 },
  { _id: { split: 'test', month: 11 }, flights: 467972 },
  { _id: { split: 'test', month: 12 }, flights: 479230 }
]
```

Partition	Months	Flights
train	1-9	4,385,712
test	10-12	1,433,367
Total		**5,819,079**

The monthly partitions are similar in size (429,191 to 520,718), so none dominates. The December partition (479,230) is exactly the set replayed through Kafka (`stream_flights` holds 479,230 documents). These counts include flights without an outcome; the model in Section 9 excludes them.

7.2 P2: Partition pruning through an index

A query on one partition should read only that partition. The index `isDelayed_1_month_1` lets MongoDB count December's delayed flights from the index alone.

▶ **MONGOSH · type at the prompt flight_delay> (inside the container)**

```
var p = db.flights.explain('executionStats').count({ isDelayed: true, month: 12 })
;({
  stage: p.queryPlanner.winningPlan.inputStage.stage,
  index: p.queryPlanner.winningPlan.inputStage.indexName,
  keysExamined: p.executionStats.totalKeysExamined,
  docsExamined: p.executionStats.totalDocsExamined
})
```

OUTPUT · what you should see (nothing to type)

```
{
  stage: 'COUNT_SCAN',
  index: 'isDelayed_1_month_1',
  keysExamined: 96773,
  docsExamined: 0
}
```

`COUNT_SCAN` with `docsExamined: 0` means the count came from the index without opening a single document; the 96,773 keys are 1.66% of the collection. (One more key than the 96,772 counted in R2 is consistent with the scan reading one key past the end of the range to find its boundary.) The same count as a full collection scan took 6,280 ms, against 33 ms with the index (`docs/EXPLAIN_RESULTS.md` , Q6). This is the MongoDB counterpart of partition pruning in Hive, where `WHERE month = 12` reads only the December directory.

7.3 P3: Horizontal partitioning (sharding) analysis

Status: design and analysis only. No sharded cluster was deployed. The designed shard key for `flights` is `{ origin: 1, date: 1 }` (`PROJECT_OVERVIEW.md` §6.3; `docs/BIGDATA_JUSTIFICATION.md`). `date` alone would send every new flight to the last chunk, a write hotspot; leading with `origin` spreads writes across airports and routes airport queries to the shards that own them. The queries below check, on the real data, whether this key would distribute well.

▶ **MONGOSH · type at the prompt flight_delay> (inside the container)**

```
db.flights.aggregate([
  { $group: { _id: '$origin', flights: { $sum: 1 } } },
  { $sort: { flights: -1 } },
  { $limit: 5 }
])
db.flights.aggregate([
  { $group: { _id: { origin: '$origin', date: '$date' }, n: { $sum: 1 } } },
  { $group: { _id: null, distinctKeys: { $sum: 1 },
              maxPerKey: { $max: '$n' }, avgPerKey: { $avg: '$n' } } }
], { allowDiskUse: true })
db.flights.stats().avgObjSize
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: 'ATL', flights: 379424 },
  { _id: 'ORD', flights: 313536 },
  { _id: 'DFW', flights: 260595 },
  { _id: 'DEN', flights: 214191 },
  { _id: 'LAX', flights: 212401 }
]
[
  {
    _id: null,
    distinctKeys: 111162,
    maxPerKey: 1168,
    avgPerKey: 52.34773573703244
  }
]
674
```

The second aggregation reads all 5.8 million documents, so it takes several seconds.

Statistic	Value
Busiest origin	ATL, 379,424 flights (6.52% of 5,819,079)
Distinct `(origin, date)` key values	111,162
Most flights for one key value	1,168
Average flights per key value	52.35
Average document size	674 bytes
Largest key value in bytes	1,168 × 674 = 787,232 bytes ≈ 0.79 MB

A chunk can only be split between distinct shard-key values, so the risk is a single value larger than a chunk (a *jumbo chunk*). The largest value is about 0.79 MB, far below MongoDB's default chunk size of 128 MB (MongoDB 6.0 and later), and the key has 111,162 distinct values. **No jumbo chunks are expected.** Even ATL spans 365 date values, so its range can be split across chunks and shards.

Effect of the design	Detail
Better	Writes spread across airports; queries with `origin` go only to the owning shards
Worse	Queries without `origin` (airline + date as in R1, pure date windows) become scatter-gather across all shards
Cost	A new index on `{ origin: 1, date: 1 }`; its size was not measured
Rejected	Hashed `_id` (even writes, but every range query is broadcast); `{ airline, date }` (only 14 leading values, uneven)

8. BDA application 2: Bucketing

Bucketing groups a continuous value into ranges, so millions of rows reduce to a few readable groups. MongoDB has two stages for it: `$bucket`, where the analyst chooses the boundaries, and `$bucketAuto`, where MongoDB chooses them so each bucket holds about the same number of documents.

8.1 B1: Arrival-delay histogram with `$bucket`

The boundaries are the project's own (`DELAY_BUCKET_BOUNDARIES` in `server/src/stats/builders.ts`). Each bucket includes its lower bound and excludes its upper bound, and the boundary at 15 matches the US DOT definition of a delayed flight. Only completed flights are counted. The second query reads the collection that `npm run build:stats` stored from the same pipeline, so the two results can be compared side by side.

▶ MONGOSH · type at the prompt `flight_delay>` (inside the container)

```
db.flights.aggregate([
  { $match: { isDelayed: { $ne: null } } },
  { $bucket: { groupBy: '$arrDelay',
    boundaries: [-100000, -15, 0, 15, 30, 60, 120, 180, 100000],
    default: 'outOfRange', output: { flights: { $sum: 1 } } } }
])
db.stats_delay_buckets.find({}, { buildId: 0 }).sort({ _id: 1 })
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: -100000, flights: 1098212 },
  { _id: -15, flights: 2402687 },
  { _id: 0, flights: 1149670 },
  { _id: 15, flights: 412835 },
  { _id: 30, flights: 325013 },
  { _id: 60, flights: 209335 },
  { _id: 120, flights: 68395 },
  { _id: 180, flights: 47861 }
]
```

Both queries print this same list: the live `$bucket` over 5.8 million documents equals the stored `stats_delay_buckets`, which serves the dashboard's `/api/analytics/delay-distribution`.

Bucket (`_id` = lower bound)	Arrival delay	Flights	Share
-100000	more than 15 min early	1,098,212	19.22%
-15	0-15 min early	2,402,687	42.05%
0	0-14 min late	1,149,670	20.12%
15	15-29 min late	412,835	7.22%
30	30-59 min late	325,013	5.69%
60	1-2 h late	209,335	3.66%

Bucket (_id = lower bound)	Arrival delay	Flights	Share
120	2-3 h late	68,395	1.20%
180	3 h or more late	47,861	0.84%
Total		**5,714,008**	**100%**

Checks. The buckets add up to 5,714,008 = 5,819,079 flights − 105,071 without an outcome (`ml/artifacts/metrics.json` , `excluded.noOutcome`); no flight fell into `outOfRange` . The four late buckets add up to 1,063,439 flights: an overall delay rate of **18.61%**.

Finding. 61.27% of completed flights arrived early, but the distribution has a long right tail: 116,256 flights (2.03%) arrived two or more hours late. An average alone would hide this.

8.2 B2: Equal-frequency distance buckets with `$bucketAuto`

`$bucketAuto` picks the boundaries so that each of 5 buckets holds about a fifth of the flights (quintiles), which suits a variable whose distribution is not known in advance.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
db.flights.aggregate([
  { $match: { isDelayed: { $ne: null } } },
  { $bucketAuto: { groupBy: '$distance', buckets: 5,
    output: { flights: { $sum: 1 },
              delayed: { $sum: { $cond: ['$isDelayed', 1, 0] } } } } },
  { $project: { flights: 1,
                delayRate: { $round: [{ $divide: ['$delayed', '$flights'] }, 4] } } }
], { allowDiskUse: true })
```

OUTPUT · what you should see (nothing to type)

```
[
  { _id: { min: 31, max: 329 }, flights: 1143907, delayRate: 0.1795 },
  { _id: { min: 329, max: 547 }, flights: 1151651, delayRate: 0.1827 },
  { _id: { min: 547, max: 822 }, flights: 1144097, delayRate: 0.186 },
  { _id: { min: 822, max: 1199 }, flights: 1145063, delayRate: 0.1959 },
  { _id: { min: 1199, max: 4983 }, flights: 1129290, delayRate: 0.1866 }
]
```

The buckets are nearly equal (1,129,290 to 1,151,651 flights) and add up to 5,714,008, matching B1. The delay rate rises from 0.1795 for the shortest flights to 0.1959 at 822-1,199 miles, then falls to 0.1866. The whole spread is only 1.6 percentage points, and distance is correlated with airline, airport and time of day, none of which is controlled for here, so this is a descriptive pattern, not a cause. It agrees with the low weight of `distance` in the trained model (importance 0.0199, third-lowest of 13 features; `ml/artifacts/metrics.json`).

8.3 Clean-up

Remove the throwaway database and confirm that only the real database is left. Then leave mongosh.

▶ MONGOSH · type at the prompt flight_delay> (inside the container)

```
demo.dropDatabase()
show dbs
exit
```

OUTPUT · what you should see (nothing to type)

```
{ ok: 1, dropped: 'flight_delay_demo' }
admin          40.00 KiB
config        132.00 KiB
flight_delay    1.35 GiB
local          80.00 KiB
```

`flight_delay_demo` is gone and `flight_delay` is untouched. The sizes shown by `show dbs` change slightly between runs. After `exit`, the prompt is `PS ...>` again.

8.4 Hive comparison (not executed)

This project does not use Hive or Hadoop, and the HiveQL below was not run. It shows how the same two ideas would be written in a Hive warehouse.

HIVEQL · for comparison only, not executed

```
-- Not executed. For comparison only.
CREATE TABLE flights (
  airline STRING, origin STRING, dest STRING, flight_date DATE,
  distance INT, arr_delay INT, is_delayed BOOLEAN
)
PARTITIONED BY (month INT)
CLUSTERED BY (origin) INTO 32 BUCKETS
STORED AS ORC;

-- The B1 histogram in HiveQL
SELECT CASE WHEN arr_delay < -15 THEN -100000 WHEN arr_delay < 0 THEN -15
            WHEN arr_delay < 15 THEN 0 WHEN arr_delay < 30 THEN 15
            WHEN arr_delay < 60 THEN 30 WHEN arr_delay < 120 THEN 60
            WHEN arr_delay < 180 THEN 120 ELSE 180 END AS bucket,
       COUNT(*) AS flights
FROM flights WHERE is_delayed IS NOT NULL
GROUP BY 1;
```

Concept	Hive	This project (MongoDB)
Partitioning	`PARTITIONED BY (month)`: one directory per month; `WHERE month = 12` reads one directory	`month` and `split` on every document + index `isDelayed_1_month_1` (P1, P2): pruning through an index, not directories
Distribution across machines	HDFS blocks on data nodes	Sharding on `{ origin, date }` (designed, analysed in P3, not deployed)
Bucketing for storage	`CLUSTERED BY (origin) INTO n BUCKETS`: rows hashed into n files	Closest analogue: hashed sharding, rejected here (P3)
Bucketing for analysis	`CASE` + `GROUP BY`, or `NTILE(5)`	`$bucket` (B1), `$bucketAuto` (B2)

Concept	Hive	This project (MongoDB)
Materialised summaries	`INSERT OVERWRITE TABLE ... SELECT`	`$group` + `$merge` with a `buildId` sweep (D2)

Why MongoDB here. MongoDB was set by the project brief (`PROJECT_OVERVIEW.md` §2), so this is a justification, not a benchmark. The workload is *load once, then serve many fast reads to an API*. Hive and Hadoop are batch engines with job start-up latency and would need a separate serving database; at 5.8 million rows (3.9 GB as BSON) a Hadoop cluster would be overhead. MongoDB answers selective reads through indexes (R1, P2) and keeps dashboard summaries fresh with `$merge`, so every measured API endpoint answered in under 1 s (`docs/EXPLAIN_RESULTS.md`). Spark would be the better choice for model training at much larger scale (`docs/BIGDATA_JUSTIFICATION.md` , "MongoDB vs alternatives").

9. Machine learning model

Task. Binary classification: will the flight arrive 15 or more minutes late (`isDelayed`)? Cancelled and diverted flights are excluded.

Split (time-based). All values below are from `ml/artifacts/metrics.json` .

Set	Period	Rows	Purpose
Train	2015-01-01 to 2015-09-30	4,299,046	Model fitting
Validation	September (month 9)	462,153	Early stopping, threshold choice, model selection
Test	2015-10-01 to 2015-12-31	1,411,835	Scored once
Excluded		105,071 without an outcome; 3,127 with an unresolved airport	

The delay rate is 19.45% in training and 16.08% in the test period (`classBalance`), so the base rate shifts between the two periods.

Features (13). `dayOfWeek` , `schedDepHour` , `isHolidayWeek` , `airline` , `origin` , `dest` , `distance` , `scheduledTime` , `airlineRate` , `originRate` , `destRate` , `originHourRate` , `routeRate` (`featureList`).

Leakage controls.

1. **Forbidden features.** `depDelay` , `taxiOut` , `taxiIn` , `airTime` , `arrDelay` , `elapsedTime` , every `causes.*` field, `cancelled` and `diverted` are post-departure information. A test rejects them, and `ml:register` refuses a run whose feature list contains them (`docs/SCHEMA.md` , "Model registry").
2. **Training-only rates.** The rate features come from `delay_lookup` , built only from the train split: `rate = (delayed + 50 × globalRate) / (completed + 50)` .
3. **Out-of-fold rates.** Training rows read fold-excluded rates (`fold1` - `fold3` exclude months 1-3, 4-6 and 7-9), so no row's feature includes its own label (`README.md` , `docs/SCHEMA.md`).
4. **Time-based split.** The split is stored on each flight (Section 7.1).

Results on the held-out test period (`ml/artifacts/metrics.json` , `models.*.test`):

Model	ROC-AUC	PR-AUC	Precision	Recall	F1
Baseline (constant train rate)	0.5000	0.1608	0	0	0
Logistic regression	0.6155	0.2218	0.2155	0.5306	0.3065
LightGBM (served)	**0.6276**	**0.2348**	**0.2183**	**0.5818**	**0.3175**

LightGBM used 157 trees (`bestIteration`). Its validation ROC-AUC was 0.6449. The decision threshold, 0.2070, was chosen on September validation data as the maximum-F1 point on the precision-recall curve, not the default 0.5.

Confusion matrix (LightGBM, test, threshold 0.2070):

	Predicted on time	Predicted delayed
Actually on time	711,981 (TN)	472,825 (FP)
Actually delayed	94,948 (FN)	132,081 (TP)

At this threshold the model flags 42.8% of test flights. A flagged flight is delayed 21.8% of the time, against a 16.1% base rate (1.36 times the base rate; derived from `metrics.json`).

Feature importance (top 5, `featureImportance`): `originHourRate` 0.344, `schedDepHour` 0.138, `airlineRate` 0.137, `airline` 0.070, `routeRate` 0.063. The strongest signals are when and where a flight departs.

Serving. The model is exported to ONNX (723,011 bytes). Its maximum absolute difference from native LightGBM over 1,411,835 rows is 2.02×10^{-7} (`onnx` in `metrics.json`). The API serves it with `onnxruntime-node`. Training took 89.5 s (`durationSeconds`). The run is registered in `model_runs` (2 runs stored; the served one is `lightgbm-20261013093139-12f165c2`).

10. Streaming layer (simulated velocity)

The speed layer replays December 2015 (479,230 flights) from the CSV into Kafka (`flights.raw`, 3 partitions, one broker, replication factor 1). The consumer (`stream:consume`) upserts each flight into `stream_flights` and scores it with the same `predictDelay` code path as `POST /api/predict`, using only pre-departure fields. December is test data, so the scores are out of sample. Offsets are committed only after the MongoDB write, which gives at-least-once delivery, and the idempotent upsert in Section 6.3 (U2) makes it effectively once. All figures below are from `docs/STREAMING.md`, as summarised in `docs/BIGDATA_JUSTIFICATION.md`:

Measure	Result
Conservation, all of December	479,230 sent = 479,230 stored = 479,230 distinct keys; consumer lag 0
Idempotent replay of 5,000 events	`stream_flights` stays at 479,230; only `deliveries` grows
Consumer throughput with live scoring	506/s at a 500/s replay; 2,018/s at 2,000/s; 2,442/s unthrottled
Latency p50 / p95	63 / 906 ms at 500/s; 138 / 2,236 ms at 2,000/s
Hard kill inside a batch	the uncommitted events were redelivered; 0 duplicates

Limits. This is a replay of 2015 data, not a live feed, and each event already carries its outcome. A single broker with replication factor 1 has no fault tolerance. Everything ran on one machine.

11. Results and discussion

Result	Value	Source
Ingest	5,819,079 rows, 0 rejected, 362 MB peak Node RSS	`ingest_runs` via `docs/BIGDATA_JUSTIFICATION.md`
Selective read via index (R1)	45,966 keys = docs = returned; 106 ms (7,037 ms as a collection scan)	`report-queries.output.txt`; `docs/EXPLAIN_RESULTS.md`
Partition-pruned count (P2)	0 documents examined, 96,773 keys, 22 ms	`report-queries.output.txt`
Computed pattern	Route ranking 13,499 ms on `flights` vs 38 ms median from `stats_route`	`docs/EXPLAIN_RESULTS.md`
Live `$bucket` vs stored histogram (B1)	identical (`true`)	`report-queries.output.txt`
Shard-key analysis (P3)	111,162 distinct values; largest ≈ 0.79 MB	`report-queries.output.txt`
Overall delay rate (completed flights)	18.61% (1,063,439 of 5,714,008)	B1
Model, test ROC-AUC	0.6276 (logistic regression 0.6155, baseline 0.5)	`ml/artifacts/metrics.json`

Discussion.

1. **Access paths matter more than raw size.** The same 5.8 million documents can be answered in milliseconds or in seconds, depending on whether the query can use an index (R1, P2) or must touch every document (Q8). Indexes help only selective queries. Whole-collection rankings, which a dashboard mostly shows, need pre-aggregation. The computed pattern turns them into reads of a few thousand documents.
2. **Partitioning serves both analytics and modelling.** One stored `split` field defines the train/test boundary for every job, which removes a whole class of leakage bugs. The month field together with a compound index gives index-only counts for a partition.
3. **Bucketing compresses millions of rows into a readable distribution.** Delays are heavily skewed: 61.27% of completed flights arrive early, and 2.03% arrive two or more hours late. A mean alone would hide this.
4. **The model is modest, and honestly so.** Delays depend mainly on information that is unavailable before departure: weather on the day, air traffic control programmes and a late inbound aircraft. These appear only in the forbidden `causes.*` fields. Logistic regression and LightGBM reach similar AUC (0.6155 and 0.6276), which suggests the limit lies in the available features, not in the algorithm.

12. Limitations

Limitation	Consequence
The data covers one year (2015)	No year-over-year signal; the test period is one season of one year, and its delay rate (16.08%) differs from training (19.45%)
No weather, air-traffic or aircraft-position/inbound-flight features	Test ROC-AUC is 0.6276, below the 0.65-0.72 range anticipated in PROJECT_OVERVIEW.md §7
Sharding is a design and analysis only	No sharded cluster was deployed; P3 analyses key distribution on a single node, and no cluster performance figures exist
Velocity is simulated	The Kafka layer replays December 2015 on one broker with replication factor 1, on one machine
Single-node measurements	Timings (for example R1: 106 ms here, 107 ms in `docs/EXPLAIN_RESULTS.md`) vary with cache state and machine load; no figure is a mean of repeated runs
Partly derived October airport codes	1,632 flights still have an unresolved origin and are excluded from airport-keyed results (`docs/SCHEMA.md`)
Hive was not used	Section 8.4 is a conceptual comparison only
Distance analysis is univariate	The B2 pattern is not adjusted for airline, airport or time

13. Conclusion and future scope

The project shows that a single-node MongoDB deployment, designed deliberately, can store, query and analyse 5.8 million flight records with interactive response times:

- **Ingestion** streams the CSV in bounded memory.
- **Denormalisation** removes joins from the hot path.
- **Indexes**, including a partial index, make selective queries cheap.
- **The computed pattern** with `$merge` makes dashboard aggregates cheap and idempotent.
- **The CRUD operations** demonstrated on real data correspond one to one with operations in the codebase.
- **Partitioning** provides both partition pruning and a leakage-safe ML split, and the shard-key analysis indicates that the designed key would distribute well.
- **Bucketing** with `$bucket` and `$bucketAuto` summarises the delay distribution and its relationship with distance. The live result reproduces the stored histogram exactly.
- **The classifier** reaches a test ROC-AUC of 0.6276 using pre-departure information only.

Future scope.

1. Deploy the designed sharded cluster (`{ origin: 1, date: 1 }`) and measure targeted versus scatter-gather queries.
2. Add more years of data to capture year-over-year patterns.

3. Join weather forecasts available before departure, and the previous leg of the same aircraft, as features that do not leak the label.
4. Update `stats_*` incrementally from the stream instead of rebuilding them in full.
5. Move training to Spark MLlib if the data grows by orders of magnitude, keeping MongoDB as the system of record.

References

1. US Department of Transportation, Bureau of Transportation Statistics. "2015 Flight Delays and Cancellations." Kaggle dataset. https://www.kaggle.com/datasets/usdot/flight-delays
2. MongoDB, Inc. *MongoDB Manual* (version 7.0): aggregation stages `$bucket`, `$bucketAuto`, `$group`, `$merge`, `$out`; partial indexes; `explain` results; sharding, shard keys and chunks. https://www.mongodb.com/docs/manual/
3. Apache Software Foundation. *Apache Hive Language Manual: DDL* (partitioned and bucketed tables). https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL
4. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." *Advances in Neural Information Processing Systems 30*.
5. ONNX Runtime. https://onnxruntime.ai/
6. Apache Software Foundation. *Apache Kafka Documentation*. https://kafka.apache.org/documentation/
7. Marz, N., and Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning (batch and speed layers).
8. Project documents: `PROJECT_OVERVIEW.md`, `README.md`, `docs/SCHEMA.md`, `docs/BIGDATA_JUSTIFICATION.md`, `docs/EXPLAIN_RESULTS.md`, `docs/STREAMING.md`, `docs/DECISIONS.md`, `docs/PROJECT_WALKTHROUGH.md`, `ml/artifacts/metrics.json`, `docs/report/viva-runbook.mongosh.js`, `docs/report/report-queries.js`, `docs/report/report-queries.output.txt`.

Appendix A: Viva quick sheet

A one-page order of play for a live demonstration. Each line says where to type.

#	Where	Type	You should see
1	PowerShell	`cd "D:\Projects\2015 Flight Delays and Cancellations"`	prompt `PS D:\Projects\2015 Flight Delays and Cancellations>`
2	PowerShell	`docker compose up -d mongo` then `docker compose ps`	`mongo` … `Up ... (healthy)`
3	PowerShell	`docker compose exec mongo mongosh flight_delay`	prompt changes to `flight_delay>`
4	mongosh	`db.flights.countDocuments()`	`5819079`
5	mongosh	Setup (6.0, step 5), then C1-C3 (6.1)	`700`, then `704`
6	mongosh	R1-R4 (6.2)	index `airline_1_date_1`, `96772`, NK `0.2995`, DCA-JFK
7	mongosh	U1-U2 (6.3)	`modifiedCount: 704`; `deliveries` 4 → 5
8	mongosh	D1-D2 (6.4)	`deletedCount: 10`, `deletedCount: 1`
9	mongosh	P1-P3 (Section 7)	12 monthly partitions; `COUNT_SCAN`, `docsExamined: 0`; 111,162 keys
10	mongosh	B1-B2 (Section 8)	the 8 delay buckets, twice; 5 distance buckets
11	mongosh	Clean-up (8.3): `demo.dropDatabase()`, `show dbs`, `exit`	no `flight_delay_demo`; prompt back to `PS ...>`

Timing. Every step answers within a few seconds except the P3 shard-key aggregation and B2, which each read all 5.8 million documents.

If you want to start the demonstration again, repeat step 5 from the Setup block: it drops `flight_delay_demo` first, so nothing is duplicated.

Appendix B: Running everything as one script

Instead of pasting block by block, the whole sequence can be run at once. These are **PowerShell** commands, run from the project folder. The first line only stores the container name in a variable, `$c`, so the second line stays short. `Get-Content` reads the file and the pipe `|` feeds it, line by line, into mongosh inside the container (`docker exec -i` keeps its input open), exactly as if you had pasted each block.

▶ **POWERSHELL · type at the prompt PS D:\Projects\2015 Flight Delays and Cancellations>**

```
$c = "2015flightdelaysandcancellations-mongo-1"
Get-Content docs/report/viva-runbook.mongosh.js | docker exec -i $c mongosh flight_delay
```

To keep a copy of the results, add `| Out-File -Encoding utf8 my-run.txt` at the end of the second line.

`docs/report/viva-runbook.mongosh.js` holds exactly the mongosh blocks of Sections 6-8, in order, and the output matches the OUTPUT blocks of this report (verified on 2026-10-27; only the ObjectIds in C3 and the database sizes in `show dbs` differ between runs). It takes about 70 s, mostly the two full-collection aggregations.

Do not use `mongosh --file` for this script: in file mode mongosh refuses shell helpers such as `show dbs` (`SyntaxError: Missing semicolon`) and does not print the result of each line. The second script, `docs/report/report-queries.js`, is written for file mode and prints labelled JSON; its saved output from 2026-10-27 is `docs/report/report-queries.output.txt`.

Git Bash instead of PowerShell. Use input redirection: `docker exec -i 2015flightdelaysandcancellations-mongo-1 mongosh flight_delay < docs/report/viva-runbook.mongosh.js`.

Appendix C: Troubleshooting during the viva

Symptom	Cause	Fix
`An expression was expected after '('.`	A MongoDB query was typed in PowerShell	Run `docker compose exec mongo mongosh flight_delay` first
`Uncaught: SyntaxError: Missing semicolon.` on a `docker ...` line	A Docker command was typed in mongosh	Type `exit`, then run it in PowerShell
`no configuration file provided: not found`	`docker compose` was run outside the project folder	`cd` into the project folder, or use the `docker exec -it 2015flightdelaysandcancellations-mongo-1 mongosh flight_delay` form
`error during connect` or `Cannot connect to the Docker daemon`	Docker Desktop is not running	Start Docker Desktop, wait until it says it is running, retry
`service "mongo" is not running`	The container is stopped	`docker compose up -d mongo`
`E11000 duplicate key error` on C2 or U2	The write examples were run twice without the setup step	Run the Setup block (6.0, step 5) again, then continue from C1
`ReferenceError: demo is not defined`	A new mongosh session was opened; `var` handles do not survive `exit`	Run the Setup block again
`db.flights.countDocuments()` returns `0`	The shell is on the wrong database (for example after `use test`)	`use flight_delay`